Linear system with Jacobi method :

Full name: khadija boumakla

Major : M1 data science and ai

Group : 2

Academic Year: 2026 – 2027

Supervised by A.EL IDRISSI

Jacobi Method

Iterative solution of linear systems Ax = b

1. Principle of the method

The Jacobi method is an **iterative** method for solving a linear system of n equations in n unknowns A x = b. Instead of computing the exact solution directly (as Gaussian elimination does), we build a sequence of vectors $x^{(0)}$, $x^{(1)}$, $x^{(2)}$, ... that converges to the solution.

In each equation i, we isolate the unknown x_i (this requires $a_{ii} \neq 0$):

$$x_i^{(k+1)} = (b_i - \text{sum}_{j!=i}\ a_{ij}\ x_j^{(k)}) / a_{ii}$$

Key point: all components of $x^{(k+1)}$ are computed only from $x^{(k)}$ (the previous iteration). This is the difference with the Gauss-Seidel method, which immediately reuses the values already updated.

Matrix form

We write A = D + L + U, where D is the diagonal of A, L its strictly lower triangular part and U its strictly upper triangular part. The iteration reads:

$$x^{(k+1)} = D^{-1} (b - (L + U) x^{(k)})$$

Convergence condition

The method converges for any initial vector $x^{(0)}$ if, for example, the matrix A is **strictly diagonally dominant**:

$$|a_{ii}| > \text{sum}_{j!=i}\ |a_{ij}| \text{ for every } i$$

More generally, the method converges if and only if the spectral radius of the iteration matrix $J = -D^{-1}(L + U)$ is strictly less than 1.

Stopping criterion

The iterations stop when the difference between two successive iterates is small enough:

$$\max_i\ | x_i^{(k+1)} - x_i^{(k)} | < \text{tol} \text{ (for example tol} = 10^{-4})$$

2. Worked example

Solve the following system with $x^{(0)} = (0, 0, 0)$:

10 x1 + x2 + x3 = 12
2 x1 + 10 x2 + x3 = 13
2 x1 + 2 x2 + 10 x3 = 14

Convergence check: row 1: 10 > 1 + 1 = 2; row 2: 10 > 2 + 1 = 3; row 3: 10 > 2 + 2 = 4. The matrix is strictly diagonally dominant, so Jacobi converges.

Iteration formulas

$$x1^{(k+1)} = (12 - x2^{(k)} - x3^{(k)}) / 10$$
$$x2^{(k+1)} = (13 - 2\ x1^{(k)} - x3^{(k)}) / 10$$
$$x3^{(k+1)} = (14 - 2\ x1^{(k)} - 2\ x2^{(k)}) / 10$$

Iteration 1 (k = 0 to 1)

x1 = (12 - 0 - 0)/10 = 1.2 x2 = (13 - 0 - 0)/10 = 1.3 x3 = (14 - 0 - 0)/10 = 1.4

Iteration 2 (k = 1 to 2)

x1 = (12 - 1.3 - 1.4)/10 = 0.93
x2 = (13 - 2×1.2 - 1.4)/10 = 0.92
x3 = (14 - 2×1.2 - 2×1.3)/10 = 0.90

Table of iterations

k	x1	x2	x3	Max difference
0	0.00000	0.00000	0.00000	-
1	1.20000	1.30000	1.40000	1.4000
2	0.93000	0.92000	0.90000	0.5000
3	1.01800	1.02400	1.03000	0.1300
4	0.99460	0.99340	0.99160	0.0384
5	1.00150	1.00192	1.00240	0.0108
6	0.99957	0.99946	0.99932	0.0031
7	1.00012	1.00015	1.00019	0.0009
8	0.99997	0.99996	0.99994	0.0003
9	1.00001	1.00001	1.00002	0.0001

Solution: with tol = 10^{-4}, the method stops at iteration 9 with x ~ (1.00001, 1.00001, 1.00002). The exact solution is **x = (1, 1, 1)**: substituting into the system confirms it (10+1+1 = 12, 2+10+1 = 13, 2+2+10 = 14).

3. Algorithm

Inputs: matrix A (n×n), vector b, initial vector $x^{(0)}$, tolerance tol, maximum number of iterations N_{max}.
Output: approximation x of the solution of A x = b.

```
Jacobi(A, b, x0, tol, Nmax)
  1. n <- size of b
  2. x <- x0
  3. FOR k = 1 TO Nmax DO
  4.     FOR i = 1 TO n DO
  5.         sum <- 0
  6.         FOR j = 1 TO n DO
  7.             IF j != i THEN
  8.                 sum <- sum + A[i][j] * x[j]
  9.             END IF
 10.         END FOR
 11.         x_new[i] <- (b[i] - sum) / A[i][i]
 12.     END FOR
 13.     IF max |x_new[i] - x[i]| < tol THEN
 14.         RETURN x_new
 15.     END IF
 16.     x <- x_new
 17. END FOR
 18. RETURN x      (not converged after Nmax iterations)
```

Remarks

- We need a_{ii} != 0 for every i; otherwise, swap the rows of A.
- Two vectors (x and x_new) must be kept: this is what distinguishes Jacobi from Gauss-Seidel.
- Cost per iteration: about n^2 operations for a dense matrix.
- The n components of x_new are independent: the method is very easy to parallelize.